

신용카드 사기 탐지 — 클래스 불균형 분류 모델

김소연

1. 문제 정의

Kaggle의 Credit Card Fraud Detection 데이터는 총 284,807건의 거래 중 사기 거래가 492건으로, 전체의 0.17%에 불과한 극단적 클래스 불균형을 가진다. 이 경우 모든 거래를 정상으로 예측하기만 해도 정확도 (Accuracy)가 99.83%에 달하지만, 이는 사기를 단 한 건도 탐지하지 못하는 무의미한 모델이다. 따라서 본 과제는 Accuracy가 아닌 Recall, F1-score, PR-AUC를 기준으로 모델을 평가한다.

변수는 Time, Amount, Class를 제외한 V1~V28이 개인정보 보호를 위해 PCA로 변환된 익명 변수이며, 결측치는 존재하지 않는다.

그림 1 Class 분포 건수·비율 출력

```
=== 건수 ===
Class
0    10000
1      492
Name: count, dtype: int64

=== 비율 ===
Class
0    0.953107
1    0.0046893
Name: proportion, dtype: float64
```

2. 데이터 샘플링

원본 데이터는 정상 거래가 28만 건에 달해 그대로 다루기에는 비효율적이고 불균형도 지나치게 극단적이다. 이에 사기 거래(492건)는 전부 유지하고 정상 거래는 10,000건만 무작위 추출(random_state=42)하여 병합했다. 그 결과 사기 비율이 0.17%에서 약 4.7%로 완화되었으나, 여전히 소수 클래스이므로 이후 SMOTE를 통한 추가 보정이 필요하다.

그림 2 샘플링 후 Class 비율

```
=== 건수 ===
Class
0    284315
1      492
Name: count, dtype: int64

=== 비율 ===
Class
0    0.998273
1    0.001727
Name: proportion, dtype: float64
```

3. 데이터 전처리

Amount 변수를 StandardScaler로 표준화하여 Amount_Scaled로 대체하고 원본 Amount는 제거했으며, 이후 특성(X)과 타겟(Class, y)을 분리했다.

다만 엄밀한 관점에서 StandardScaler는 학습 데이터로만 fit해야 테스트 데이터의 분포 정보가 학습에 유입되는 전처리 누수를 방지할 수 있다. 본 과제에서는 명세 흐름에 따라 분할 전 스케일링을 수행했으나, 최종 모델로 선택한 트리 기반 모델은 변수의 스케일이 아닌 임계값을 기준으로 분할하므로 이 스케일링이 성능에 미치는 영향은 제한적이다.

4. 학습·테스트 데이터 분할

train_test_split으로 8:2 비율로 분할하되, stratify=y 옵션을 적용해 클래스 비율을 유지했다. 그 결과 학습 데이터의 사기 비율은 4.69%, 테스트 데이터는 4.67%로 거의 일치하여, 평가가 특정 분할의 우연에 좌우되지 않도록 했다. 이후 테스트 데이터는 최종 평가 전까지 학습 과정에 일절 사용하지 않았다.

```
=== Train Class 비율 ===
Class
0    0.953056
1    0.046944
Name: proportion, dtype: float64
건수:
Class
0    7999
1     394
Name: count, dtype: int64

=== Test Class 비율 ===
Class
0    0.953311
1    0.046689
Name: proportion, dtype: float64
건수:
Class
0    2001
1      98
Name: count, dtype: int64
```

5. SMOTE 적용

적용 이유: 학습 데이터의 사기 거래는 394건으로 정상 7,999건에 비해 크게 부족하다. 이 상태로 학습하면 모델이 다수 클래스(정상)에 편향되어 사기의 패턴을 충분히 학습하지 못한다. SMOTE는 소수 클래스 샘플 사이를 보간해 합성 사기 데이터를 생성하고 두 클래스의 개수를 맞추므로써, 모델이 사기의 결정 경계를 제대로 학습하도록 돕는다.

적용 범위: SMOTE는 학습 데이터(X_train)에만 적용했다. 테스트 데이터에까지 적용하면 테스트 주변 정보를 이용해 합성 샘플이 생성되어 데이터 누수가 발생하기 때문이다. 그 결과 사기 거래가 394건에서 7,999건으로 증강되어 정상과 1:1 균형을 이루었다.

6. 모델 학습 및 결과

모델 선택: RandomForest를 선정했다. 익명화된 V1~V28 변수들은 타겟과 비선형 관계를 가질 수 있고, 트리 기반 모델은 스케일에 민감하지 않으며, 복잡한 튜닝 없이도 안정적인 성능을 내는 비선형 baseline으로 적합하기 때문이다.

증강된 학습 데이터로 모델을 학습하고, 원본 테스트 데이터에서 예측값과 예측 확률을 산출했다. 목표 대비 결과는 다음과 같다.

지표	목표	Class 0	Class 1
Recall	≥ 0.80	0.9975	0.8878
F1-score	≥ 0.88	0.9960	0.9158
PR-AUC	≥ 0.90	0.9537	

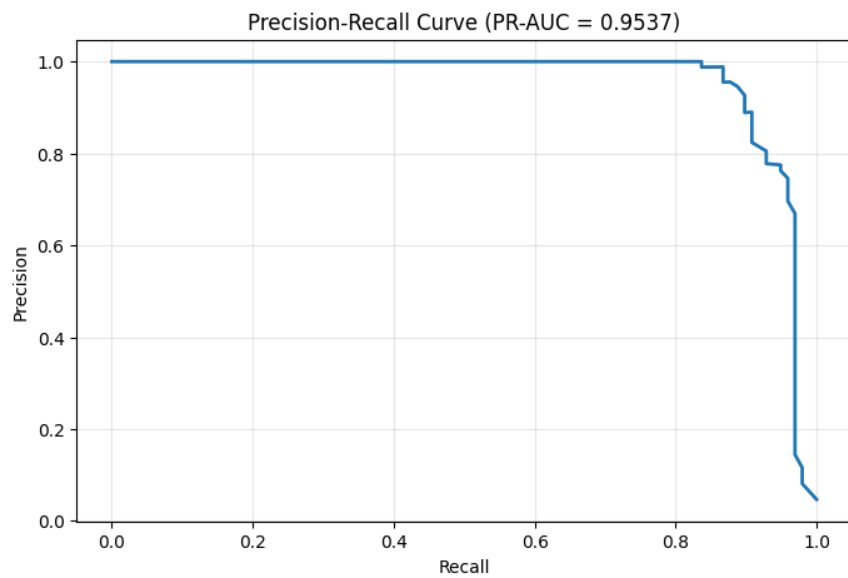
세 목표 모두 두 클래스에서 달성했다. 별도의 하이퍼파라미터 튜닝이나 threshold 조정 없이 기본 설정만으로 목표를 충족했다. 테스트셋의 사기 98건 중 87건을 탐지했으며(Recall 0.888), 사기로 예측한 건 중 94.6%가 실제 사기(Precision 0.946)로 정밀도와 재현율의 균형이 우수하다.

7. Threshold 분석

기본 threshold(0.5)로 모든 목표를 달성했으나, 실제 사기 탐지는 오분류 비용이 비대칭적이다. 이를 고려해 threshold 조정에 따른 성능 변화를 관찰했다.

Threshold	Precision	Recall	F1
0.5	0.9263	0.8980	0.9119
0.4	0.8318	0.9082	0.8683
0.3	0.7344	0.9592	0.8319
0.2	0.5621	0.9694	0.7116
0.1	0.3532	0.9694	0.5177

threshold를 낮출수록 Recall은 0.898에서 0.969까지 상승하지만 Precision은 0.926에서 0.353으로 급락한다. 이는 사기를 놓치지 않으려면(Recall $\uparrow$) 정상 거래를 사기로 오판하는 비용(Precision $\downarrow$)을 감수해야 함을 보여준다. F1-score 기준으로는 기본 threshold(0.5)가 최적이며, threshold를 낮출수록 오히려 F1이 하락한다.



Precision-Recall 곡선은 Recall 0.8 부근까지 Precision 1.0을 유지하다 이후 완만히 하락하며, 이는 PR-AUC 0.954의 우수한 성능을 뒷받침한다.

결론: 최적 threshold는 평가 지표가 아니라 비즈니스 비용이 결정한다. 사기 한 건을 놓치는 손실이 오탐 처리 비용보다 크다면, F1을 일부 희생하더라도 threshold를 0.3~0.4로 낮춰 Recall을 확보하는 선택이 정당화된다. 본 과제에서는 모든 목표를 이미 충족했으므로 기본 threshold를 최종 설정으로 채택했다.